Intro

This notebook analyzes global migration patterns from 1990 to 2024, focusing on how male and female migration patterns around the world. Using Random Forest machine learning models, it predicts future migration flows and identifies key patterns.

The work includes exploring migration trends over time, creating features from historical data, comparing different prediction models (Random Forest, XGBoost, SVR), fine-tuning model settings, and testing how well the models work on both numbers and categories. The study reveals insights about gender differences in migration, finds the most important factors for predictions, and tests how well models can forecast future migration using data split by time periods.

Loading required libraries for data processing, visualization, and machine learning

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
%pip install category_encoders
from category_encoders import TargetEncoder

from sklearn.svm import SVR
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor

from scipy.stats import randint
from sklearn.model_selection import RandomizedSearchCV, TimeSeriesSplit

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, confusion_matrix, classification_report

import warnings
warnings.filterwarnings("ignore")
```

```
Collecting category_encoders
  Downloading category_encoders-2.8.1-py3-none-any.whl.metadata (7.9 kB)
Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.12/dist-packages (from category_encoders) (2.0.2)
Requirement already satisfied: pandas>=1.0.5 in /usr/local/lib/python3.12/dist-packages (from category_encoders) (2.2.2)
Requirement already satisfied: patsy>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from category_encoders) (1.0.1)
Requirement already satisfied: scikit-learn>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from category_encoders) (1.6.1)
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from category_encoders) (1.16.1)
Requirement already satisfied: statsmodels>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from category_encoders) (0.14.5)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.5->category_encoc
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.5->category_encoders) (2025
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.5->category_encoders) (20
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=1.6.0->category_encoders
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=1.6.0->category_e
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.12/dist-packages (from statsmodels>=0.9.0->category_encoder
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=1.0.5->cate
Downloading category_encoders-2.8.1-py3-none-any.whl (85 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 85.7/85.7 kB 2.5 MB/s eta 0:00:00
Installing collected packages: category_encoders
Successfully installed category_encoders-2.8.1
```

Understanding the dataset

Loading the raw migration dataset and exploring basic information

```
df_raw = pd.read_csv('FL_Migration_Data.csv', index_col=0)

print(f"Data loaded with shape: {df_raw.shape}")
df_raw.head()
```

```
Data loaded with shape: (28030, 30)
```

Index	Region, development group, country or area of destination	Coverage	Data type	Location code of destination	Region, development group, country or area of origin	Location code of origin	1990	1995	2000	2005	...	2020.1
1	World	NaN	NaN	900	World	900	153916063	163176002	174566152	192788721	...	143223497
2	World	NaN	NaN	900	Sub-Saharan Africa	1834	14124662	15183742	14584913	16004417	...	14375255
3	World	NaN	NaN	900	Northern Africa and Western Asia	1833	14986109	17216219	18728264	21198002	...	21212741
4	World	NaN	NaN	900	Central and Southern Asia	1831	30342957	27930630	30008559	32445580	...	30263324
5	World	NaN	NaN	900	Eastern and South-Eastern Asia	1832	14465509	17262816	20822011	24315849	...	17926170

5 rows × 30 columns

Displaying detailed dataset information

```
df_raw.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 28030 entries, 1 to 28030
Data columns (total 30 columns):
 #   Column                                                  Non-Null Count  Dtype
---  ------                                                  --------------  -----
 0   Region, development group, country or area of destination  28030 non-null  object
 1   Coverage                                                2416 non-null   float64
 2   Data type                                               17163 non-null  object
 3   Location code of destination                            28030 non-null  int64
 4   Region, development group, country or area of origin    28030 non-null  object
 5   Location code of origin                                 28030 non-null  int64
 6   1990                                                    28030 non-null  int64
 7   1995                                                    28030 non-null  int64
 8   2000                                                    28030 non-null  int64
 9   2005                                                    28030 non-null  int64
 10  2010                                                    28030 non-null  int64
 11  2015                                                    28030 non-null  int64
 12  2020                                                    28030 non-null  int64
 13  2024                                                    28030 non-null  int64
 14  1990.1                                                  28030 non-null  int64
 15  1995.1                                                  28030 non-null  int64
 16  2000.1                                                  28030 non-null  int64
 17  2005.1                                                  28030 non-null  int64
 18  2010.1                                                  28030 non-null  int64
 19  2015.1                                                  28030 non-null  int64
 20  2020.1                                                  28030 non-null  int64
 21  2024.1                                                  28030 non-null  int64
 22  1990.2                                                  28030 non-null  int64
 23  1995.2                                                  28030 non-null  int64
 24  2000.2                                                  28030 non-null  int64
 25  2005.2                                                  28030 non-null  int64
 26  2010.2                                                  28030 non-null  int64
 27  2015.2                                                  28030 non-null  int64
 28  2020.2                                                  28030 non-null  int64
 29  2024.2                                                  28030 non-null  int64
dtypes: float64(1), int64(26), object(3)
memory usage: 6.6+ MB
```

```
df_raw.columns
```

```
Index(['Region, development group, country or area of destination', 'Coverage',
       'Data type', 'Location code of destination',
       'Region, development group, country or area of origin',
       'Location code of origin', '1990', '1995', '2000', '2005', '2010',
       '2015', '2020', '2024', '1990.1', '1995.1', '2000.1', '2005.1',
       '2010.1', '2015.1', '2020.1', '2024.1', '1990.2', '1995.2', '2000.2',
       '2005.2', '2010.2', '2015.2', '2020.2', '2024.2'],
      dtype='object')
```

```
df_raw['Region, development group, country or area of origin'].unique().tolist()
```

```
'Central America',
'Belize',
'Costa Rica',
'El Salvador',
'Guatemala',
'Honduras',
'Mexico',
'Nicaragua',
'Panama',
'South America',
'Argentina',
'Bolivia (Plurinational State of)',
'Brazil',
'Chile',
'Colombia',
'Ecuador',
'Falkland Islands (Malvinas)*',
'French Guiana*',
'Guyana',
'Paraguay',
'Peru',
'Suriname',
'Uruguay',
'Venezuela (Bolivarian Republic of)',
'NORTHERN AMERICA',
'Bermuda*',
'Canada',
'Greenland*',
'Saint Pierre and Miquelon*',
'United States of America*',
'OCEANIA',
'Australia*',
'New Zealand*',
'Melanesia',
'Fiji',
'New Caledonia*',
'Papua New Guinea',
'Solomon Islands',
'Vanuatu',
'Micronesia',
'Guam*',
'Kiribati',
'Marshall Islands',
'Micronesia (Fed. States of)',
'Nauru',
'Northern Mariana Islands*',
'Palau',
'Polynesia*',
'American Samoa*',
'Cook Islands*',
'French Polynesia*',
'Niue*',
'Samoa',
'Tokelau*',
'Tonga',
'Tuvalu',
'Wallis and Futuna Islands*',
'Others']
```

```
df_raw['Region, development group, country or area of destination'].unique().tolist()
```

```
'Greenland*'
```

```
        'Greenland*',
        'Saint Pierre and Miquelon*',
        'United States of America*',
        'OCEANIA',
        'Australia*',
        'New Zealand*',
        'Melanesia',
        'Fiji',
        'New Caledonia*',
        'Papua New Guinea',
        'Solomon Islands',
        'Vanuatu',
        'Micronesia',
        'Guam*',
        'Kiribati',
        'Marshall Islands',
        'Micronesia (Fed. States of)',
        'Nauru',
        'Northern Mariana Islands*',
        'Palau',
        'Polynesia*',
        'American Samoa*',
        'Cook Islands*',
        'French Polynesia*',
        'Niue*',
        'Samoa',
        'Tokelau*',
        'Tonga',
        'Tuvalu',
        'Wallis and Futuna Islands*']
```

Preprocessing

Filtering dataset to exclude regional aggregates, territories, and non-sovereign entities to focus on country-to-country migration flows

```
exclude_list = [
    'World',
    'More developed regions',
    'Less developed regions',
    'Least developed countries',
    'Less developed regions, excluding least developed countries',
    'Less developed regions, excluding China',
    'Land-locked Developing Countries (LLDC)',
    'Small Island Developing States (SIDS)',
    'High-and-upper-middle-income countries',
    'Low-and-Lower-middle-income countries',
    'High-income countries',
    'Low-and-middle-income countries',
    'Middle-income countries',
    'Upper-middle-income countries',
    'Lower-middle-income countries',
    'Low-income countries',
    'No income group available',
    'AFRICA',
    'Eastern Africa',
    'Middle Africa',
    'Northern Africa',
    'Southern Africa',
    'Western Africa',
    'ASIA',
    'Central Asia',
    'Eastern Asia',
    'Southern Asia',
    'South-Eastern Asia',
    'Western Asia',
    'EUROPE',
    'Eastern Europe',
    'Northern Europe',
    'Southern Europe',
    'Western Europe',
    'LATIN AMERICA AND THE CARIBBEAN',
    'Caribbean',
    'Central America',
    'South America',
    'NORTHERN AMERICA',
    'OCEANIA',
    'Melanesia',
    'Micronesia',
    'Polynesia',
    'Europe and Northern America',
    'Oceania (excluding Australia and New Zealand)',
    'Australia/New Zealand',
    'Mayotte',
```

```
        'Réunion',
        'Saint Helena',
        'China, Hong Kong SAR',
        'China, Macao SAR',
        'China, Taiwan Province of China',
        "Dem. People's Republic of Korea",
        'State of Palestine',
        'Western Sahara',
        'Channel Islands*',
        'Faroe Islands*',
        'Gibraltar*',
        'Holy See*',
        'Isle of Man*',
        'Monaco',
        'San Marino',
        'Andorra',
        'Bermuda*',
        'Greenland*',
        'Saint Pierre and Miquelon*',
        'Anguilla*',
        'Aruba*',
        'Bonaire, Sint Eustatius and Saba*',
        'British Virgin Islands*',
        'Cayman Islands*',
        'Curaçao*',
        'Guadeloupe*',
        'Martinique*',
        'Montserrat*',
        'Puerto Rico*',
        'Sint Maarten (Dutch part)*',
        'Turks and Caicos Islands*',
        'United States Virgin Islands*',
        'Falkland Islands (Malvinas)*',
        'French Guiana*',
        'New Caledonia*',
        'Guam*',
        'Northern Mariana Islands*',
        'Tokelau*',
        'Wallis and Futuna Islands*',
        'Central and Southern Asia',
        'Eastern and South-Eastern Asia',
        'Latin America and the Caribbean',
        'Northern Africa and Western Asia',
        'Sub-Saharan Africa',
        'American Samoa*',
        'Cook Islands*',
        'French Polynesia*',
        'Niue*',
        'Polynesia*',
        'China, Taiwan Province of China',
        'State of Palestine',
        'Others'
    ]

    df_filtered = df_raw[
        (~df_raw['Region, development group, country or area of origin'].isin(exclude_list)) &
        (~df_raw['Region, development group, country or area of destination'].isin(exclude_list))
    ].copy().reset_index(drop=True)

    df_filtered
```

	Region, development group, country or area of destination	Coverage	Data type	Location code of destination	Region, development group, country or area of origin	Location code of origin	1990	1995	2000	2005	...	2020.1	2024.1	1990.2	199
0	Burundi	NaN	B R	108	Kenya	404	186	216	202	502	...	693	779	82	
1	Burundi	NaN	B R	108	Rwanda	646	221943	160197	66415	61104	...	34728	38992	113171	82
2	Burundi	NaN	B R	108	Uganda	800	1833	1546	949	805	...	619	695	935	
3	Burundi	NaN	B R	108	United Republic of Tanzania	834	11912	9780	5692	13732	...	15535	17443	6074	5
4	Burundi	NaN	B R	108	Democratic Republic of the Congo	180	36654	41447	37951	99099	...	101757	114251	18691	21
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
8203	Tuvalu	NaN	C	798	United States of America*	840	41	21	11	7	...	3	3	4	
8204	Tuvalu	NaN	C	798	Fiji	242	114	81	60	54	...	34	35	49	
8205	Tuvalu	NaN	C	798	Kiribati	296	63	61	54	53	...	12	12	50	
8206	Tuvalu	NaN	C	798	Nauru	520	7	6	5	5	...	2	2	6	
8207	Tuvalu	NaN	C	798	Samoa	882	21	11	6	4	...	2	2	3	

8208 rows × 30 columns

```
df_filtered.columns
```

```
Index(['Region, development group, country or area of destination', 'Coverage',
       'Data type', 'Location code of destination',
       'Region, development group, country or area of origin',
       'Location code of origin', '1990', '1995', '2000', '2005', '2010',
       '2015', '2020', '2024', '1990.1', '1995.1', '2000.1', '2005.1',
       '2010.1', '2015.1', '2020.1', '2024.1', '1990.2', '1995.2', '2000.2',
       '2005.2', '2010.2', '2015.2', '2020.2', '2024.2'],
      dtype='object')
```

Transforming dataset from wide to long format and parse year columns to separate total, male, and female migration data

```
id_vars = [
    'Region, development group, country or area of destination',
    'Location code of destination',
    'Region, development group, country or area of origin',
    'Location code of origin'
]

value_vars = [col for col in df_filtered.columns if col not in id_vars and col not in ['Coverage', 'Data type']] # Exclude already known

df_long = df_filtered.melt(
    id_vars=id_vars,
    value_vars=value_vars,
    var_name='Year_Raw',  # Temporary column to hold original year string (e.g., '1990', '1990.1', '1990.2')
    value_name='Migrant_Count' # Temporary column to hold the migrant count
)

def parse_year_and_type(year_raw_string):
    if '.' in year_raw_string:
        year_part, type_suffix = year_raw_string.split('.')
        year = int(year_part)
        if type_suffix == '1':
            migrant_type = 'Male_Migrants'
        elif type_suffix == '2':
            migrant_type = 'Female_Migrants'
        else:
            migrant_type = 'Unknown'
    else:
        year = int(year_raw_string)
        migrant_type = 'Total_Migrants'
    return year, migrant_type

df_long[['Year', 'Migrant_Type']] = df_long['Year_Raw'].apply(lambda x: pd.Series(parse_year_and_type(x)))

df_transformed = df_long.pivot_table(
    index=[
        'Region, development group, country or area of destination',
```

```
        'Location code of destination',
        'Region, development group, country or area of origin',
        'Location code of origin',
        'Year'
    ],
    columns='Migrant_Type',
    values='Migrant_Count'
).reset_index()

df_transformed = df_transformed.rename(columns={
    'Region, development group, country or area of destination': 'Destination_Country',
    'Region, development group, country or area of origin': 'Origin_Country'
})

df_transformed = df_transformed[[ 'Origin_Country', 'Destination_Country', 'Year', 'Male_Migrants', 'Female_Migrants', 'Total_Migrants']]

df_transformed['Origin_Country'] = df_transformed['Origin_Country'].str.replace('*', '', regex=False).str.strip()
df_transformed['Destination_Country'] = df_transformed['Destination_Country'].str.replace('*', '', regex=False).str.strip()
df_transformed = df_transformed.sort_values(by=['Origin_Country', 'Destination_Country', 'Year']).copy()

print("\nDataFrame Info:")
print(df_transformed.info())

df_transformed.head()
```

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Index: 65664 entries, 1288 to 65615
Data columns (total 6 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Origin_Country       65664 non-null  object
 1   Destination_Country  65664 non-null  object
 2   Year                 65664 non-null  int64
 3   Male_Migrants        65664 non-null  float64
 4   Female_Migrants      65664 non-null  float64
 5   Total_Migrants       65664 non-null  float64
dtypes: float64(3), int64(1), object(2)
memory usage: 3.5+ MB
None
```

Migrant_Type	Origin_Country	Destination_Country	Year	Male_Migrants	Female_Migrants	Total_Migrants
1288	Afghanistan	Australia	1990	2207.0	1722.0	3929.0
1289	Afghanistan	Australia	1995	3358.0	2979.0	6337.0
1290	Afghanistan	Australia	2000	7325.0	5186.0	12511.0
1291	Afghanistan	Australia	2005	10743.0	8176.0	18919.0
1292	Afghanistan	Australia	2010	17088.0	13675.0	30763.0

```
row_to_check = df_transformed.sample(1)
print(f"\nSample row for verification:\n{row_to_check.to_string(index=False)}")
print(f"Males + Females: {row_to_check['Male_Migrants'].iloc[0] + row_to_check['Female_Migrants'].iloc[0]}")
print(f"Total: {row_to_check['Total_Migrants'].iloc[0]}")
```

```
Sample row for verification:
Origin_Country Destination_Country  Year  Male_Migrants  Female_Migrants  Total_Migrants
         China      United Kingdom  2015       118503.0         135568.0        254071.0
Males + Females: 254071.0
Total: 254071.0
```

EDA

Visualizing temporal migration patterns to understand the overall trends and gender-specific flows from 1990 to 2024

```
migration_by_year_gender = df_transformed.groupby('Year')[['Total_Migrants', 'Male_Migrants', 'Female_Migrants']].sum().reset_index()

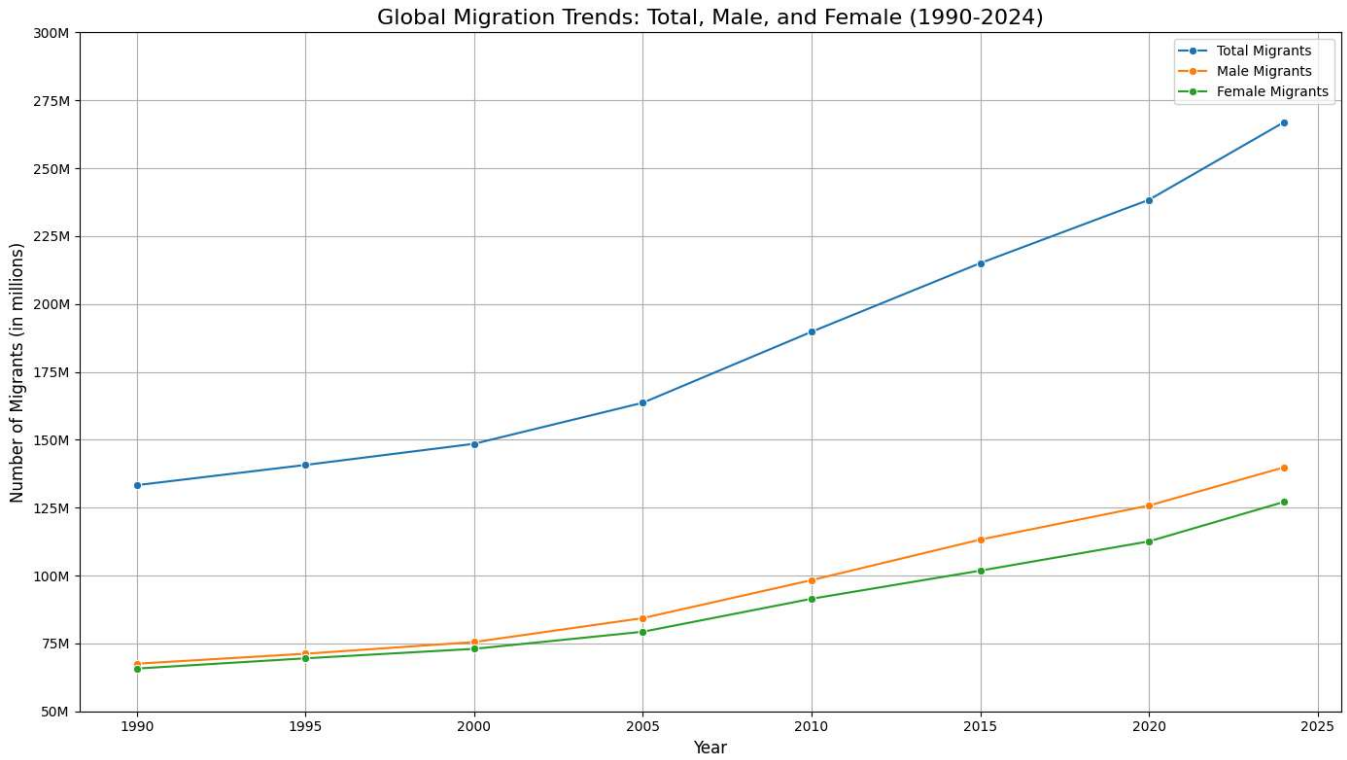
plt.figure(figsize=(14, 8))
sns.lineplot(data=migration_by_year_gender, x='Year', y='Total_Migrants', marker='o', label='Total Migrants')
sns.lineplot(data=migration_by_year_gender, x='Year', y='Male_Migrants', marker='o', label='Male Migrants')
sns.lineplot(data=migration_by_year_gender, x='Year', y='Female_Migrants', marker='o', label='Female Migrants')

plt.title('Global Migration Trends: Total, Male, and Female (1990-2024)', fontsize=16)
plt.xlabel('Year', fontsize=12)
plt.ylabel('Number of Migrants (in millions)', fontsize=12)

current_ticks = plt.yticks()[0]
```

```
plt.yticks(current_ticks, [f'{int(tick / 1_000_000)}M' for tick in current_ticks])

plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

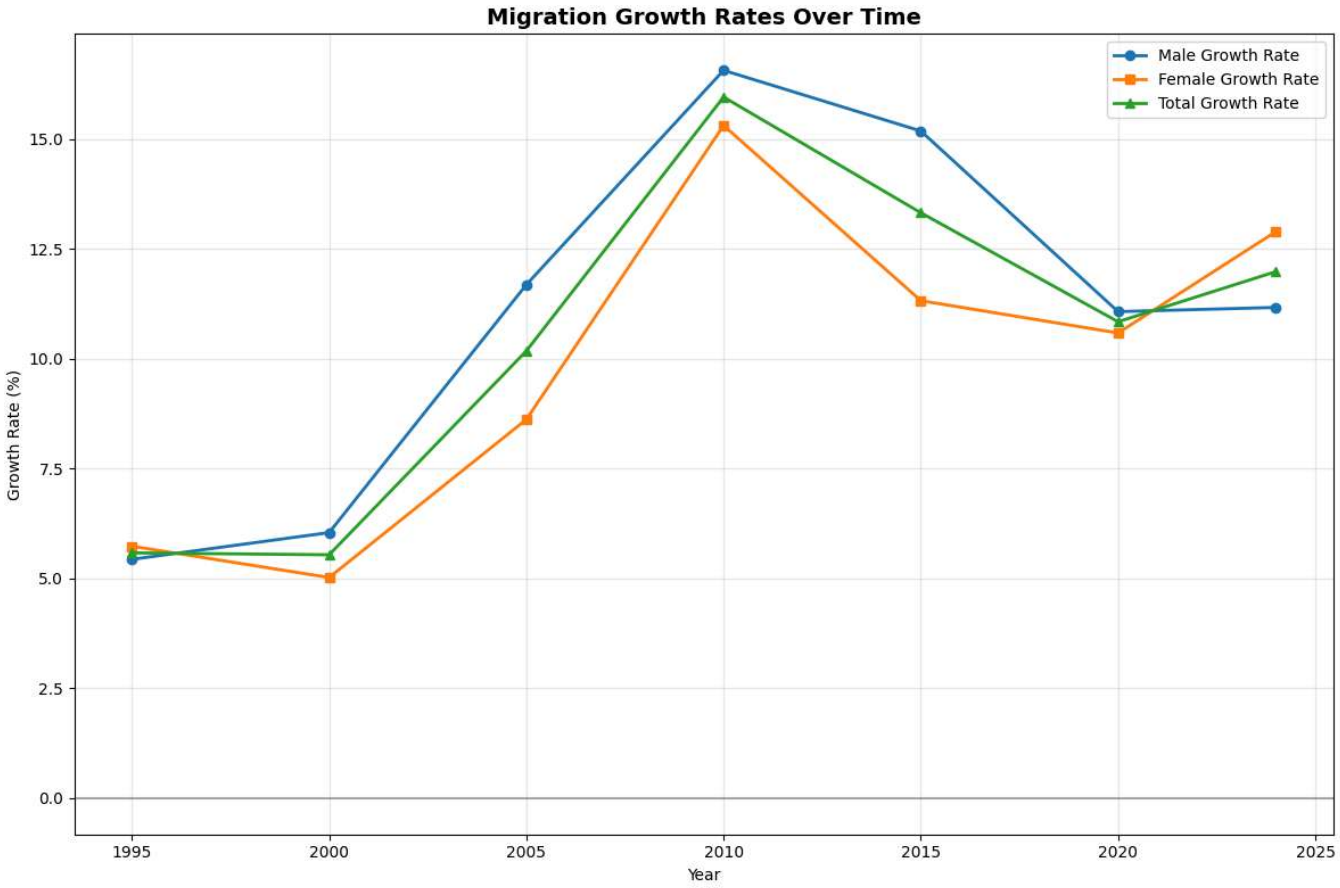


Analyzing year-over-year growth rates to identify periods of acceleration or deceleration in migration trends

```
yearly_totals = df_transformed.groupby('Year')[['Male_Migrants', 'Female_Migrants', 'Total_Migrants']].sum()
yearly_totals['Male_Growth'] = yearly_totals['Male_Migrants'].pct_change() * 100
yearly_totals['Female_Growth'] = yearly_totals['Female_Migrants'].pct_change() * 100
yearly_totals['Total_Growth'] = yearly_totals['Total_Migrants'].pct_change() * 100

plt.figure(figsize=(12, 8))
plt.plot(yearly_totals.index, yearly_totals['Male_Growth'], marker='o', label='Male Growth Rate', linewidth=2)
plt.plot(yearly_totals.index, yearly_totals['Female_Growth'], marker='s', label='Female Growth Rate', linewidth=2)
plt.plot(yearly_totals.index, yearly_totals['Total_Growth'], marker='^', label='Total Growth Rate', linewidth=2)

plt.axhline(y=0, color='black', linestyle='-', alpha=0.3)
plt.title('Migration Growth Rates Over Time', fontsize=14, fontweight='bold')
plt.xlabel('Year')
plt.ylabel('Growth Rate (%)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```




```
import matplotlib.ticker as mticker

# Destination Top 10 by Year (1990–2024)
dest_data = df_transformed[(df_transformed['Year'] >= 1990) & (df_transformed['Year'] <= 2024)].copy()
dest_data['Total_Migrants_Million'] = dest_data['Total_Migrants'] / 1_000_000

dest_trends = (
    dest_data.groupby(['Year', 'Destination_Country'], as_index=False)['Total_Migrants_Million']
    .sum()
)

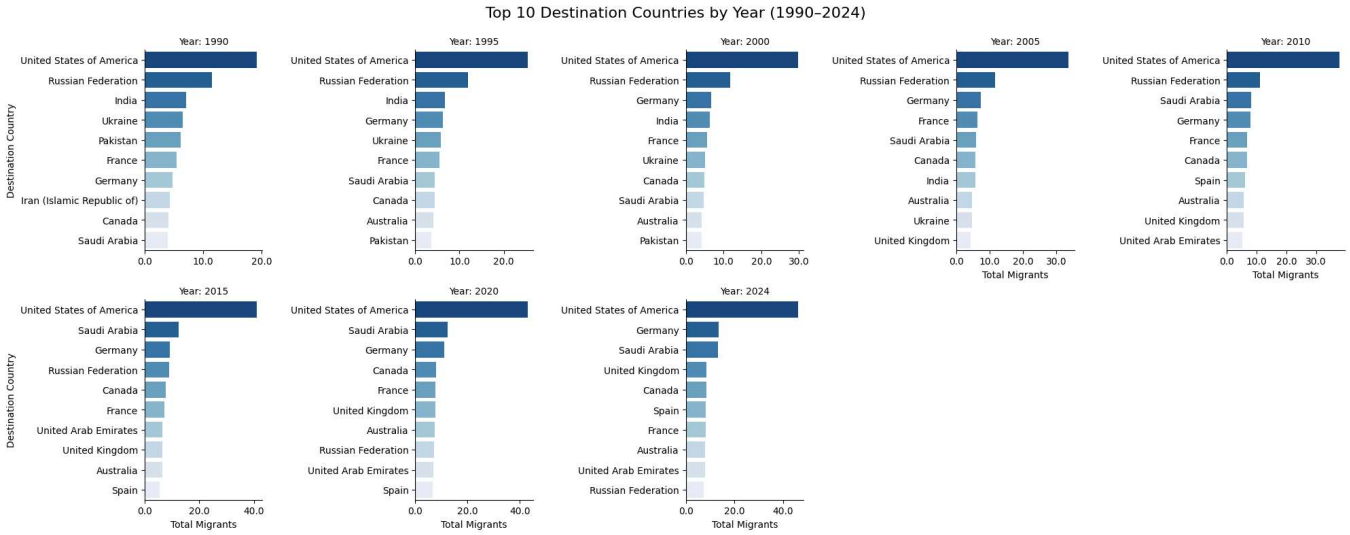
top_dest_per_year = (
    dest_trends.groupby('Year')
    .apply(lambda x: x.nlargest(10, 'Total_Migrants_Million'))
    .reset_index(drop=True)
)
g = sns.FacetGrid(top_dest_per_year, col="Year", col_wrap=5, height=4, sharex=False, sharey=False)

def barplot_with_sort(data, **kwargs):
    order = data.sort_values("Total_Migrants_Million", ascending=False)["Destination_Country"]
    sns.barplot(
        data=data,
        x="Total_Migrants_Million",
        y="Destination_Country",
        order=order,
        palette="Blues_r",
        **kwargs
    )

g.map_dataframe(barplot_with_sort)

for ax in g.axes.flat:
    ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"{x:.1f}"))

g.set_titles("Year: {col_name}")
g.set_axis_labels("Total Migrants", "Destination Country")
plt.subplots_adjust(top=0.90)
g.fig.suptitle("Top 10 Destination Countries by Year (1990–2024)", fontsize=16)
plt.show()
```




```
import matplotlib.ticker as mticker

# Origin Top 10 by Year (1990–2024)
origin_data = df_transformed[(df_transformed['Year'] >= 1990) & (df_transformed['Year'] <= 2024)].copy()
origin_data['Total_Migrants_Million'] = origin_data['Total_Migrants'] / 1_000_000

origin_trends = (
    origin_data.groupby(['Year', 'Origin_Country'], as_index=False)['Total_Migrants_Million']
    .sum()
)

top_origin_per_year = (
    origin_trends.groupby('Year')
    .apply(lambda x: x.nlargest(10, 'Total_Migrants_Million'))
    .reset_index(drop=True)
)

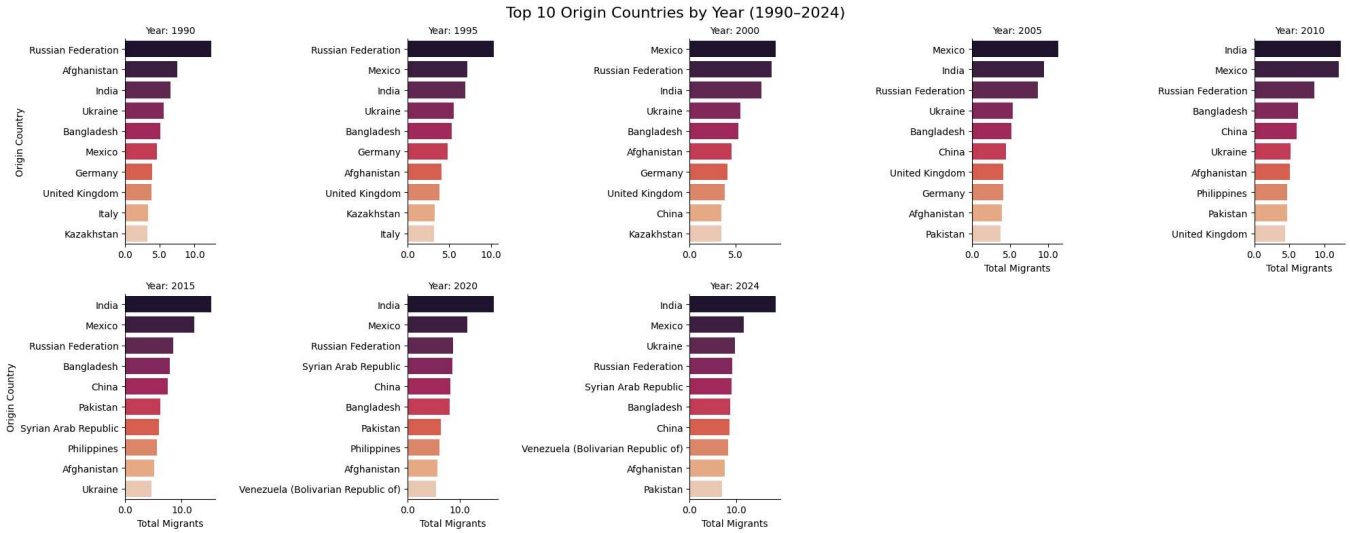
g = sns.FacetGrid(top_origin_per_year, col="Year", col_wrap=5, height=4, sharex=False, sharey=False)

def barplot_with_sort(data, **kwargs):
    order = data.sort_values("Total_Migrants_Million", ascending=False)["Origin_Country"]
    sns.barplot(
        data=data,
        x="Total_Migrants_Million",
        y="Origin_Country",
        order=order,
        palette="rocket",
        **kwargs
    )

g.map_dataframe(barplot_with_sort)

for ax in g.axes.flat:
    ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"{x:.1f}"))

g.set_titles("Year: {col_name}")
g.set_axis_labels("Total Migrants", "Origin Country")
plt.subplots_adjust(top=0.92)
g.fig.suptitle("Top 10 Origin Countries by Year (1990–2024)", fontsize=16)
plt.show()
```



Identifying the largest migration source countries by aggregating total migrant flows across the entire study period

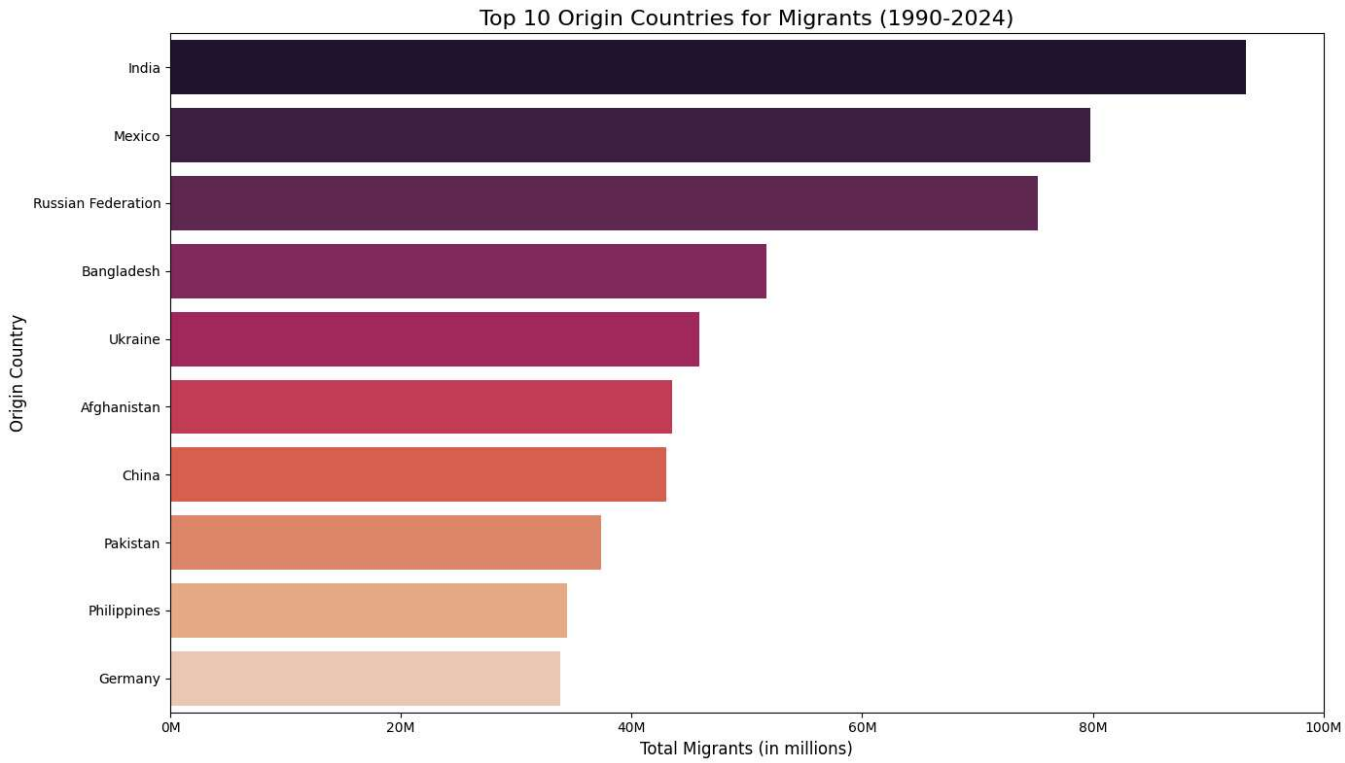
```
origin_migrants_total = df_transformed.groupby('Origin_Country')['Total_Migrants'].sum().reset_index()

top_10_origins = origin_migrants_total.sort_values(by='Total_Migrants', ascending=False).head(10)

plt.figure(figsize=(14, 8))
sns.barplot(data=top_10_origins, x='Total_Migrants', y='Origin_Country', palette='rocket', hue='Origin_Country', legend=False)

plt.title('Top 10 Origin Countries for Migrants (1990-2024)', fontsize=16)
plt.xlabel('Total Migrants (in millions)', fontsize=12)
plt.ylabel('Origin Country', fontsize=12)

current_xticks = plt.xticks()[0]
plt.xticks(current_xticks, [f'{int(tick / 1_000_000)}M' for tick in current_xticks])
plt.tight_layout()
plt.show()
```



Examining the most attractive destination countries by aggregating total migrant inflows across the entire study period

```
destination_migrants_total = df_transformed.groupby('Destination_Country')['Total_Migrants'].sum().reset_index()

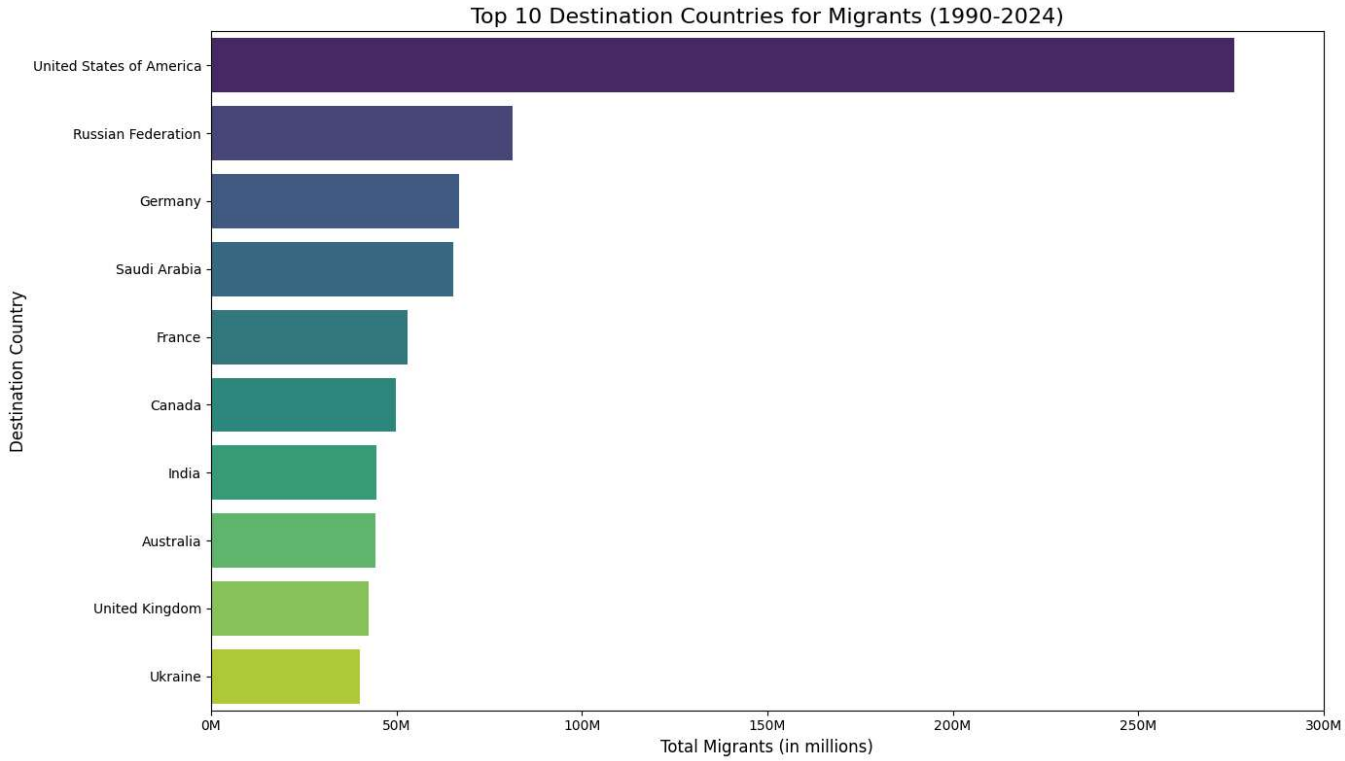
top_10_destinations = destination_migrants_total.sort_values(by='Total_Migrants', ascending=False).head(10)

plt.figure(figsize=(14, 8))
sns.barplot(data=top_10_destinations, x='Total_Migrants', y='Destination_Country', palette='viridis', hue='Destination_Country', legend=

plt.title('Top 10 Destination Countries for Migrants (1990-2024)', fontsize=16)
plt.xlabel('Total Migrants (in millions)', fontsize=12)

plt.ylabel('Destination Country', fontsize=12)

current_xticks = plt.xticks()[0]
plt.xticks(current_xticks, [f'{int(tick / 1_000_000)}M' for tick in current_xticks])
plt.tight_layout()
plt.show()
```



Exploring the busiest migration corridors by examining specific origin-destination pairs and revealing gender composition patterns across the top migration routes

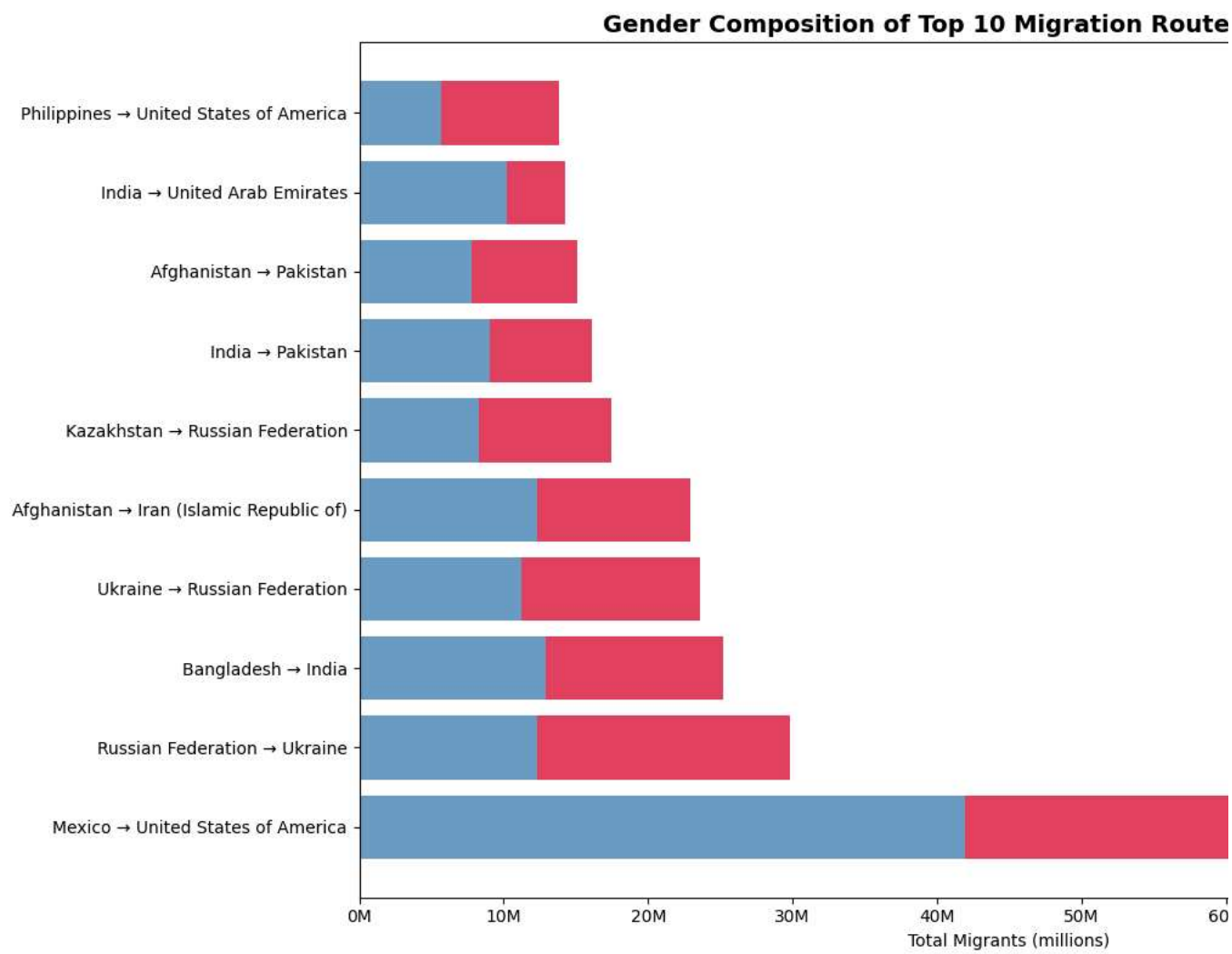
```
top_routes = df_transformed.groupby(['Origin_Country', 'Destination_Country']).agg({
    'Male_Migrants': 'sum',
    'Female_Migrants': 'sum',
    'Total_Migrants': 'sum'
}).reset_index()

top_routes = top_routes.nlargest(10, 'Total_Migrants')
top_routes['Route'] = top_routes['Origin_Country'] + ' → ' + top_routes['Destination_Country']

plt.figure(figsize=(14, 8))
plt.barh(top_routes['Route'], top_routes['Male_Migrants'], label='Male', alpha=0.8, color='steelblue')
plt.barh(top_routes['Route'], top_routes['Female_Migrants'], left=top_routes['Male_Migrants'], label='Female', alpha=0.8, color='crimson'

plt.xlabel('Total Migrants (millions)')
plt.title('Gender Composition of Top 10 Migration Routes (1990-2024)', fontsize=14, fontweight='bold')
plt.legend()

current_xticks = plt.xticks()[0]
plt.xticks(current_xticks, [f'{int(tick / 1_000_000)}M' for tick in current_xticks])
plt.tight_layout()
plt.show()
```



Computing descriptive statistics and variability metrics to quantify gender differences in migration patterns and volatility across the study period

```
gender_migration_yearly = df_transformed.groupby('Year')[['Male_Migrants', 'Female_Migrants']].sum().reset_index()

gender_stats = gender_migration_yearly[['Male_Migrants', 'Female_Migrants']].describe().transpose()

print("\nDescriptive Statistics for Male and Female Migrants (Yearly Totals):\n")
print(gender_stats.to_string())

male_mean = gender_stats.loc['Male_Migrants']['mean']
male_std = gender_stats.loc['Male_Migrants']['std']
female_mean = gender_stats.loc['Female_Migrants']['mean']
female_std = gender_stats.loc['Female_Migrants']['std']

print(f"\nMale migrants average: {male_mean:,.0f}")
print(f"Female migrants average: {female_mean:,.0f}")
print(f"Gender difference: {((male_mean - female_mean) / female_mean * 100):.1f}% more male migrants")

print(f"\nMale migration volatility (std dev): {male_std:,.0f}")
print(f"Female migration volatility (std dev): {female_std:,.0f}")
print(f"Male volatility is {((male_std - female_std) / female_std * 100):.1f}% higher")

male_cv = male_std / male_mean
female_cv = female_std / female_mean
print(f"\nMale coefficient of variation: {male_cv:.3f}")
print(f"Female coefficient of variation: {female_cv:.3f}")
```

```
Descriptive Statistics for Male and Female Migrants (Yearly Totals):

                 count         mean           std         min          25%         50%          75%          max
Migrant_Type
Male_Migrants      8.0  96961234.25  2.694819e+07  67530473.0  74427469.75  91315161.5  1.163669e+08  139821558.0
Female_Migrants    8.0  90059372.75  2.216253e+07  65745787.0  72132261.50  85370290.0  1.044923e+08  127095081.0

Male migrants average: 96,961,234
Female migrants average: 90,059,373
```

```
Gender difference: 7.7% more male migrants

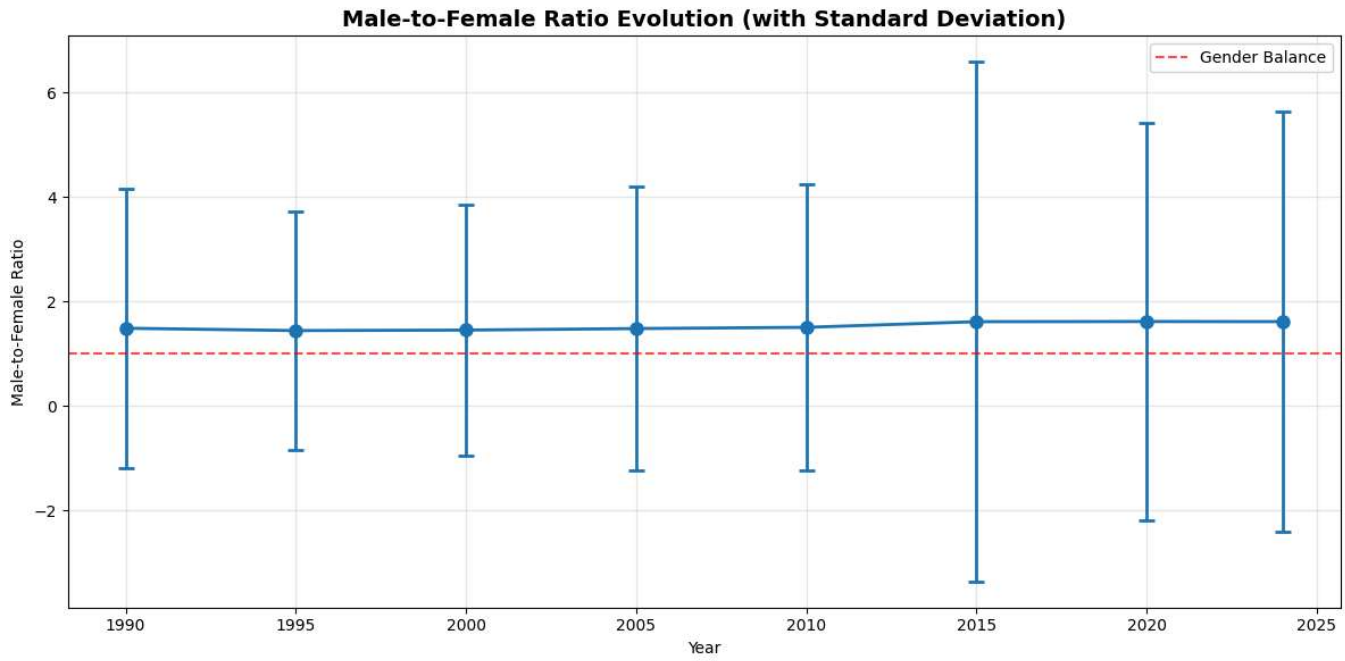
Male migration volatility (std dev): 26,948,185
Female migration volatility (std dev): 22,162,533
Male volatility is 21.6% higher

Male coefficient of variation: 0.278
Female coefficient of variation: 0.246
```

Analyzing temporal changes in gender balance by visualizing male-to-female migration ratios with variability measures across years

```
df_stats = df_transformed.copy()
df_stats['Male_to_Female_Ratio'] = df_stats['Male_Migrants'] / df_stats['Female_Migrants'].replace(0, np.nan)
ratio_by_year = df_stats.groupby('Year')['Male_to_Female_Ratio'].agg(['mean', 'std']).reset_index()

plt.figure(figsize=(12, 6))
plt.errorbar(ratio_by_year['Year'], ratio_by_year['mean'], yerr=ratio_by_year['std'],
             marker='o', capsize=5, capthick=2, linewidth=2, markersize=8)
plt.axhline(y=1.0, color='red', linestyle='--', alpha=0.7, label='Gender Balance')
plt.xlabel('Year')
plt.ylabel('Male-to-Female Ratio')
plt.title('Male-to-Female Ratio Evolution (with Standard Deviation)', fontsize=14, fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Examining correlations between male and female migration flows through visualization and statistical analysis to understand gender-specific migration relationships across different routes and time periods

```
correlation_male_female = df_transformed['Male_Migrants'].corr(df_transformed['Female_Migrants'])

corr_vars = ['Male_Migrants', 'Female_Migrants', 'Total_Migrants', 'Year']
correlation_matrix = df_transformed[corr_vars].corr()

plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='RdBu_r', center=0, square=True, fmt='.3f', cbar_kws={'label': 'Correlation'})
plt.title('Correlation Matrix: Migration Variables', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

country_correlations = df_transformed.groupby(['Origin_Country', 'Destination_Country']).apply(
    lambda x: x['Male_Migrants'].corr(x['Female_Migrants']) if len(x) > 2 else np.nan
).reset_index()
country_correlations.columns = ['Origin_Country', 'Destination_Country', 'Male_Female_Corr']
country_correlations = country_correlations.dropna()

print("\n" + "="*60)
```

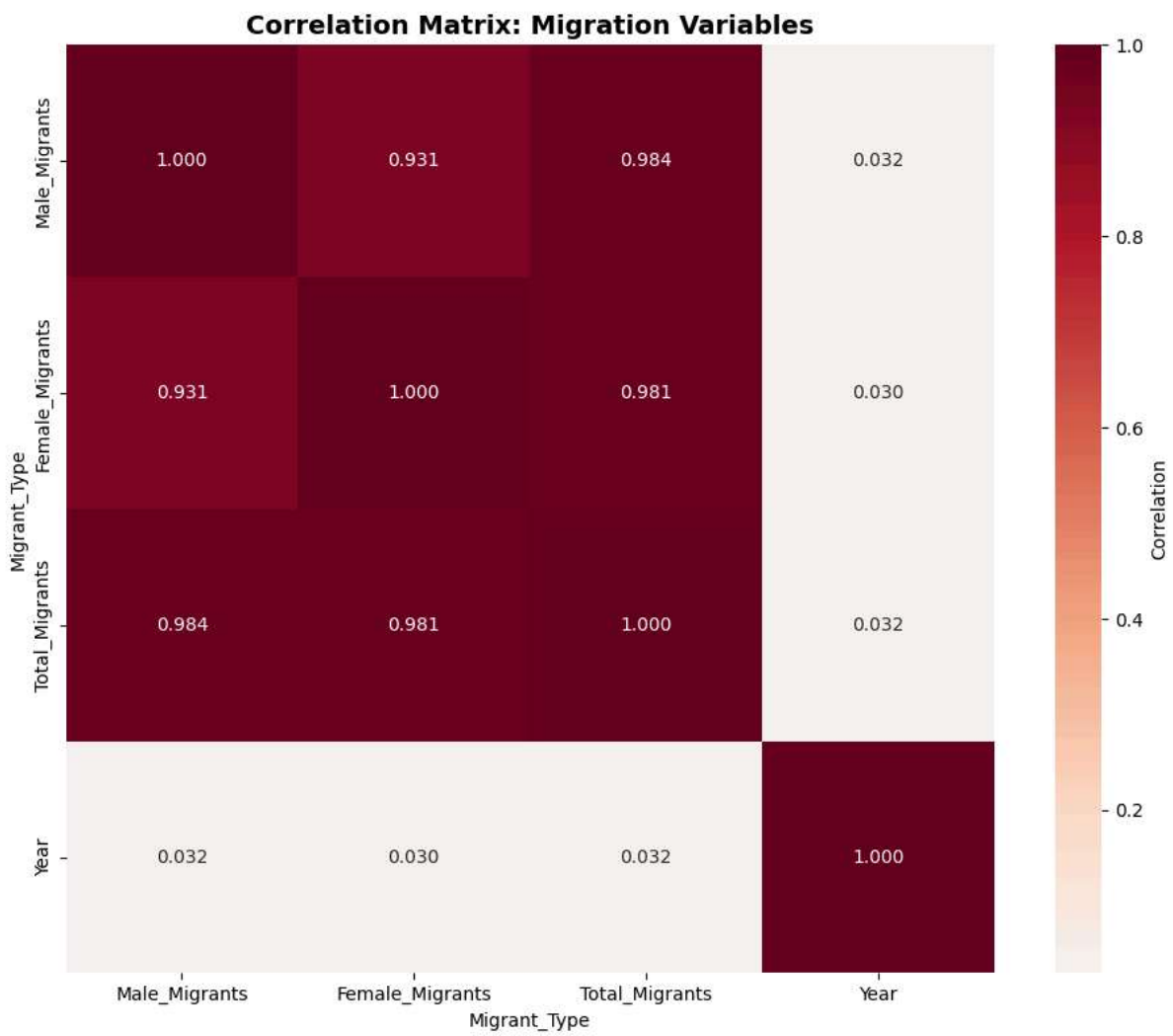
```
print("STATISTICAL ANALYSIS")
print("="*60)

print(f"1. CORRELATION ANALYSIS:")
print(f"   Overall male-female correlation: {correlation_male_female:.3f}")

print(f"\n2. VARIATION ACROSS ROUTES:")
print(f"   Mean correlation across routes: {country_correlations['Male_Female_Corr'].mean():.3f}")
print(f"   Standard deviation: {country_correlations['Male_Female_Corr'].std():.3f}")
print(f"   Range: {country_correlations['Male_Female_Corr'].min():.3f} to {country_correlations['Male_Female_Corr'].max():.3f}")

low_corr_routes = country_correlations[country_correlations['Male_Female_Corr'] < 0.5]
print(f"   {len(low_corr_routes)} routes with correlation < 0.5 (weak relationship)")

print(f"\n3. TEMPORAL VARIATION:")
ratio_std = ratio_by_year['std'].mean()
print(f"   Average std deviation of gender ratio: {ratio_std:.3f}")
```




```
============================================================
STATISTICAL ANALYSIS
============================================================
1. CORRELATION ANALYSIS:
   Overall male-female correlation: 0.931

2. VARIATION ACROSS ROUTES:
   Mean correlation across routes: 0.845
   Standard deviation: 0.358
   Range: -0.999 to 1.000
   788 routes with correlation < 0.5 (weak relationship)

3. TEMPORAL VARIATION:
   Average std deviation of gender ratio: 3.198
```

Feature Engineering

Creating temporal lag features and derived metrics including historical migration values, gender percentages, growth indicators, and change rates to capture time-series patterns for predictive modeling

```
columns_to_lag = ['Male_Migrants', 'Female_Migrants', 'Total_Migrants']

for col in columns_to_lag:
    df_transformed[f'{col}_lag_1'] = df_transformed.groupby(['Origin_Country', 'Destination_Country'])[col].shift(1)
    df_transformed[f'{col}_lag_2'] = df_transformed.groupby(['Origin_Country', 'Destination_Country'])[col].shift(2)

lagged_columns = [col for col in df_transformed.columns if '_lag_' in col]
df_transformed[lagged_columns] = df_transformed[lagged_columns].fillna(0)

df_transformed['Male_Percentage_lag_1'] = np.where(
    df_transformed['Total_Migrants_lag_1'] == 0,
    0,
    df_transformed['Male_Migrants_lag_1'] / df_transformed['Total_Migrants_lag_1']
)

df_transformed['Female_Percentage_lag_1'] = np.where(
    df_transformed['Total_Migrants_lag_1'] == 0,
    0,
    df_transformed['Female_Migrants_lag_1'] / df_transformed['Total_Migrants_lag_1']
)

df_transformed['Is_Growing_Route'] = np.where(df_transformed['Total_Migrants_lag_1'] > df_transformed['Total_Migrants_lag_2'], 1, 0)

for col in columns_to_lag:
    df_transformed[f'{col}_Change_Rate'] = np.where(
        df_transformed[f'{col}_lag_2'] == 0,
        0,
        (df_transformed[f'{col}_lag_1'] - df_transformed[f'{col}_lag_2']) / df_transformed[f'{col}_lag_2']
    )
```

Creating additional categorical features to identify top destination countries, popular migration routes, and gender-dominated flows for enhanced modeling and classification analysis

```
destination_totals = df_transformed.groupby(['Destination_Country', 'Year'])['Total_Migrants'].sum().reset_index()
destination_totals['Is_Popular_Destination'] = destination_totals.groupby('Year')['Total_Migrants'].transform(
    lambda x: x.rank(pct=True, ascending=True) >= 0.90
).astype(int)

df_transformed = df_transformed.merge(
    destination_totals[['Destination_Country', 'Year', 'Is_Popular_Destination']],
    on=['Destination_Country', 'Year'],
    how='left'
)

df_transformed['Is_Popular_Migration_Route'] = df_transformed.groupby('Year')['Total_Migrants'].transform(
    lambda x: x.rank(pct=True, ascending=True) >= 0.90
).astype(int)

df_transformed['Is_Male_Dominated'] = (df_transformed['Male_Migrants'] > df_transformed['Female_Migrants']).astype(int)
```

Displaying enhanced dataset structure and rows after feature engineering to verify all lag features, categorical indicators, and derived metrics

```
print(df_transformed.info())

df_transformed.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 65664 entries, 0 to 65663
Data columns (total 21 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   Origin_Country               65664 non-null  object
 1   Destination_Country          65664 non-null  object
 2   Year                         65664 non-null  int64
 3   Male_Migrants                65664 non-null  float64
 4   Female_Migrants              65664 non-null  float64
 5   Total_Migrants               65664 non-null  float64
 6   Male_Migrants_lag_1          65664 non-null  float64
 7   Male_Migrants_lag_2          65664 non-null  float64
 8   Female_Migrants_lag_1        65664 non-null  float64
 9   Female_Migrants_lag_2        65664 non-null  float64
 10  Total_Migrants_lag_1         65664 non-null  float64
 11  Total_Migrants_lag_2         65664 non-null  float64
 12  Male_Percentage_lag_1        65664 non-null  float64
 13  Female_Percentage_lag_1      65664 non-null  float64
 14  Is_Growing_Route             65664 non-null  int64
 15  Male_Migrants_Change_Rate    65664 non-null  float64
 16  Female_Migrants_Change_Rate  65664 non-null  float64
 17  Total_Migrants_Change_Rate   65664 non-null  float64
 18  Is_Popular_Destination       65664 non-null  int64
 19  Is_Popular_Migration_Route   65664 non-null  int64
 20  Is_Male_Dominated            65664 non-null  int64
dtypes: float64(14), int64(5), object(2)
memory usage: 10.5+ MB
None
```

	Origin_Country	Destination_Country	Year	Male_Migrants	Female_Migrants	Total_Migrants	Male_Migrants_lag_1	Male_Migrants_lag
0	Afghanistan	Australia	1990	2207.0	1722.0	3929.0	0.0	
1	Afghanistan	Australia	1995	3358.0	2979.0	6337.0	2207.0	
2	Afghanistan	Australia	2000	7325.0	5186.0	12511.0	3358.0	220
3	Afghanistan	Australia	2005	10743.0	8176.0	18919.0	7325.0	335
4	Afghanistan	Australia	2010	17088.0	13675.0	30763.0	10743.0	732

5 rows × 21 columns

Examining class distributions and balance across binary categorical features to assess data representation for modeling and classification tasks

```
print("\nValue counts for 'Is_Male_Dominated':")
print(df_transformed['Is_Male_Dominated'].value_counts().to_string(index=False))

print("\nValue counts for 'Is_Popular_Migration_Route':")
print(df_transformed['Is_Popular_Migration_Route'].value_counts().to_string(index=False))

print("\nValue counts for 'Is_Popular_Destination':")
print(df_transformed['Is_Popular_Destination'].value_counts().to_string(index=False))
```

```
Value counts for 'Is_Male_Dominated':
Is_Male_Dominated
34824
30840

Value counts for 'Is_Popular_Migration_Route':
Is_Popular_Migration_Route
59096
 6568

Value counts for 'Is_Popular_Destination':
Is_Popular_Destination
57944
 7720
```

Model Input Preparation

```
df_transformed.columns
```

```
Index(['Origin_Country', 'Destination_Country', 'Year', 'Male_Migrants',
       'Female_Migrants', 'Total_Migrants', 'Male_Migrants_lag_1',
       'Male_Migrants_lag_2', 'Female_Migrants_lag_1', 'Female_Migrants_lag_2',
       'Total_Migrants_lag_1', 'Total_Migrants_lag_2', 'Male_Percentage_lag_1',
       'Female_Percentage_lag_1', 'Is_Growing_Route',
       'Male_Migrants_Change_Rate', 'Female_Migrants_Change_Rate',
       'Total_Migrants_Change_Rate', 'Is_Popular_Destination',
```

```
       'Is_Popular_Migration_Route', 'Is_Male_Dominated'],
      dtype='object')
```

Defining predictor features including country identifiers, temporal variables, lag features, and derived metrics for machine learning model input preparation

```
predictor_columns = [
    'Origin_Country',
    'Destination_Country',
    'Year',
    'Is_Growing_Route',
    'Male_Percentage_lag_1', 'Female_Percentage_lag_1',
    'Male_Migrants_lag_1', 'Female_Migrants_lag_1', 'Total_Migrants_lag_1',
    'Male_Migrants_lag_2', 'Female_Migrants_lag_2', 'Total_Migrants_lag_2',
    'Male_Migrants_Change_Rate', 'Female_Migrants_Change_Rate', 'Total_Migrants_Change_Rate'
]

CATEGORICAL_FEATURES = ['Origin_Country', 'Destination_Country']
NUMERICAL_FEATURES = [col for col in predictor_columns if col not in CATEGORICAL_FEATURES]

X = df_transformed[predictor_columns]
y = df_transformed['Total_Migrants']

print("\n--- Features (X) Info ---")
print(X.info())
```

```
--- Features (X) Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 65664 entries, 0 to 65663
Data columns (total 15 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   Origin_Country               65664 non-null  object
 1   Destination_Country          65664 non-null  object
 2   Year                         65664 non-null  int64
 3   Is_Growing_Route             65664 non-null  int64
 4   Male_Percentage_lag_1        65664 non-null  float64
 5   Female_Percentage_lag_1      65664 non-null  float64
 6   Male_Migrants_lag_1          65664 non-null  float64
 7   Female_Migrants_lag_1        65664 non-null  float64
 8   Total_Migrants_lag_1         65664 non-null  float64
 9   Male_Migrants_lag_2          65664 non-null  float64
 10  Female_Migrants_lag_2        65664 non-null  float64
 11  Total_Migrants_lag_2         65664 non-null  float64
 12  Male_Migrants_Change_Rate    65664 non-null  float64
 13  Female_Migrants_Change_Rate  65664 non-null  float64
 14  Total_Migrants_Change_Rate   65664 non-null  float64
dtypes: float64(11), int64(2), object(2)
memory usage: 7.5+ MB
None
```

Creating temporal train-test split with historical data (<=2020) for training and future data (2024) for evaluation to simulate real-world forecasting scenarios

```
X_train = X[df_transformed['Year'] <= 2020].copy()
y_train = y[df_transformed['Year'] <= 2020].copy()

X_test = X[df_transformed['Year'] == 2024].copy()
y_test = y[df_transformed['Year'] == 2024].copy()

print(f"X_train shape: {X_train.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
X_train shape: (57456, 15)
y_train shape: (57456,)
X_test shape: (8208, 15)
y_test shape: (8208,)
```

Applying target encoding to categorical features using training data to transform country names into numerical representations based on migration patterns

```
encoder = TargetEncoder(cols=CATEGORICAL_FEATURES, min_samples_leaf=20, smoothing=10.0)

encoder.fit(X_train, y_train)

X_train_encoded = encoder.transform(X_train)
```

```
X_test_encoded = encoder.transform(X_test)

print("\nShape of X_train_encoded:", X_train_encoded.shape)
print("Shape of X_test_encoded:", X_test_encoded.shape)

print("\nFirst 5 rows of X_train_encoded (showing encoded categorical features):")
X_train_encoded.head()
```

```
Shape of X_train_encoded: (57456, 15)
Shape of X_test_encoded: (8208, 15)

First 5 rows of X_train_encoded (showing encoded categorical features):
```

	Origin_Country	Destination_Country	Year	Is_Growing_Route	Male_Percentage_lag_1	Female_Percentage_lag_1	Male_Migrants_lag_1
0	77960.512987	29639.086851	1990	0	0.000000	0.000000	0.0
1	77960.512987	29639.086851	1995	1	0.561721	0.438279	2207.0
2	77960.512987	29639.086851	2000	1	0.529904	0.470096	3358.0
3	77960.512987	29639.086851	2005	1	0.585485	0.414515	7325.0
4	77960.512987	29639.086851	2010	1	0.567842	0.432158	10743.0

Standardizing features using training data to normalize numerical values with zero mean and unit variance for machine learning algorithms requiring scaled inputs: SVM

```
scaler = StandardScaler()
scaler.fit(X_train_encoded)

X_train_scaled = scaler.transform(X_train_encoded)
X_test_scaled = scaler.transform(X_test_encoded)

X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X_train_encoded.columns, index=X_train_encoded.index)
X_test_scaled_df = pd.DataFrame(X_test_scaled, columns=X_test_encoded.columns, index=X_test_encoded.index)

print("\nDescriptive statistics for X_train_scaled_df (mean should be ~0, std dev ~1):")
print(X_train_scaled_df.describe().transpose().to_string())

print("\nFirst 5 rows of X_train_scaled_df (features are now scaled):")
X_train_scaled_df.head()
```

```
Descriptive statistics for X_train_scaled_df (mean should be ~0, std dev ~1):
                            count          mean       std       min       25%       50%       75%         max
Origin_Country              57456.0 -1.051172e-16  1.000009 -0.857751 -0.595071 -0.264270  0.314937    6.653827
Destination_Country         57456.0  4.436564e-17  1.000009 -0.330231 -0.316110 -0.287868 -0.119044    8.786675
Year                        57456.0  0.000000e+00  1.000009 -1.500000 -1.000000  0.000000  1.000000    1.500000
Is_Growing_Route            57456.0 -5.799996e-17  1.000009 -1.179032 -1.179032  0.848154  0.848154    0.848154
Male_Percentage_lag_1       57456.0  2.028762e-16  1.000009 -1.550219 -0.721436  0.266524  0.593808    2.179305
Female_Percentage_lag_1     57456.0 -6.678033e-17  1.000009 -1.479349 -1.105622  0.302950  0.653469    2.590678
Male_Migrants_lag_1         57456.0 -6.307031e-18  1.000009 -0.110612 -0.110599 -0.109764 -0.099635   79.127282
Female_Migrants_lag_1       57456.0 -3.539976e-18  1.000009 -0.114790 -0.114776 -0.114008 -0.104241   78.260321
Total_Migrants_lag_1        57456.0  8.471209e-18  1.000009 -0.114136 -0.114123 -0.113276 -0.102851   78.897900
Male_Migrants_lag_2         57456.0  1.468549e-17  1.000009 -0.098535 -0.098535 -0.098264 -0.091931   90.628117
Female_Migrants_lag_2       57456.0  1.156289e-17  1.000009 -0.102374 -0.102374 -0.102141 -0.096026   85.814459
Total_Migrants_lag_2        57456.0 -4.668440e-18  1.000009 -0.101475 -0.101475 -0.101204 -0.094749   89.291155
Male_Migrants_Change_Rate   57456.0  5.109005e-18  1.000009 -0.078325 -0.026284 -0.026284 -0.015634  214.769888
Female_Migrants_Change_Rate 57456.0 -2.767055e-18  1.000009 -0.044782 -0.016305 -0.016305 -0.010610  234.636936
Total_Migrants_Change_Rate  57456.0  4.421105e-18  1.000009 -0.057952 -0.021471 -0.021471 -0.013644  203.591190

First 5 rows of X_train_scaled_df (features are now scaled):
```

	Origin_Country	Destination_Country	Year	Is_Growing_Route	Male_Percentage_lag_1	Female_Percentage_lag_1	Male_Migrants_lag_1
0	2.268355	0.1291	-1.5	-1.179032	-1.550219	-1.479349	-0.110612
1	2.268355	0.1291	-1.0	0.848154	0.544731	0.304460	-0.083114
2	2.268355	0.1291	-0.5	0.848154	0.426070	0.433955	-0.068774
3	2.268355	0.1291	0.0	0.848154	0.633361	0.207739	-0.019349
4	2.268355	0.1291	0.5	0.848154	0.567561	0.279546	0.023236

Model Performance Comparison and Baseline Validation

Comparing baseline performance across Random Forest, XGBoost, and SVR models using standardized metrics to establish model selection benchmarks for migration prediction

```
models = {
    'RandomForestRegressor': RandomForestRegressor(random_state=42),
    'XGBRegressor': XGBRegressor(random_state=42, objective='reg:squarederror', eval_metric='rmse'),
    'SVR': SVR()
}

performance_metrics = {}

print("--- Baseline Model Comparison for Total_Migrants Prediction ---")

for name, model in models.items():
    print(f"\nTraining {name}...")

    if name.lower() == 'svr':
        current_X_train = X_train_scaled_df
        current_X_test = X_test_scaled_df
    else:
        current_X_train = X_train_encoded
        current_X_test = X_test_encoded

    model.fit(current_X_train, y_train)

    y_pred = model.predict(current_X_test)

    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)

    performance_metrics[name] = { 'MAE': mae, 'RMSE': rmse, 'R2': r2 }

    print(f"{name} Performance on Test Set (2024 data):")
    print(f"  Mean Absolute Error (MAE): {mae:,.2f}")
    print(f"  Root Mean Squared Error (RMSE): {rmse:,.2f}")
    print(f"  R-squared (R2): {r2:.4f}")

print("\n--- Summary of All Baseline Models ---")
metrics_df = pd.DataFrame.from_dict(performance_metrics, orient='index')
print(metrics_df.to_string())
```

```
--- Baseline Model Comparison for Total_Migrants Prediction ---

Training RandomForestRegressor...
RandomForestRegressor Performance on Test Set (2024 data):
  Mean Absolute Error (MAE): 4,365.37
  Root Mean Squared Error (RMSE): 36,826.90
  R-squared (R2): 0.9682

Training XGBRegressor...
XGBRegressor Performance on Test Set (2024 data):
  Mean Absolute Error (MAE): 6,813.96
  Root Mean Squared Error (RMSE): 56,137.88
  R-squared (R2): 0.9261

Training SVR...
SVR Performance on Test Set (2024 data):
  Mean Absolute Error (MAE): 32,197.20
  Root Mean Squared Error (RMSE): 208,890.88
  R-squared (R2): -0.0231

--- Summary of All Baseline Models ---
                                MAE           RMSE        R2
RandomForestRegressor   4365.368303   36826.902549  0.968203
XGBRegressor            6813.958844   56137.877008  0.926112
SVR                    32197.200044  208890.877683 -0.023060
```

Hyperparameter Tuning with Random Forest

Configuring hyperparameter search parameters for Random Forest optimization using randomized search with cross-validation settings

```
random_search_parameters = {
    'param_distributions': {
        'n_estimators': randint(100, 250),
        'max_features': ['sqrt', 'log2'],
        'max_depth': randint(15, 30),
        'min_samples_split': randint(5, 12),
        'min_samples_leaf': randint(2, 6),
    },
    'n_iter': 30,
    'scoring': 'neg_root_mean_squared_error',
    'verbose': 1,
    'random_state': 42,
```

```
    'n_jobs': 1,
    'return_train_score': True,
    'refit': True
}
```

Performing hyperparameter optimization for male migration prediction using randomized search with time series cross-validation to find optimal Random Forest parameters

```
y_train_males = df_transformed[df_transformed['Year'] <= 2020]['Male_Migrants'].copy()

rf_regressor = RandomForestRegressor(random_state=42)
tscv = TimeSeriesSplit(n_splits=3)

random_search_males = RandomizedSearchCV(estimator=rf_regressor, cv=tscv, **random_search_parameters)

random_search_males.fit(X_train_encoded, y_train_males)

print("\n--- RandomizedSearchCV for male migrants Complete ---")

print(f"\nBest parameters for Males_Migrants: {random_search_males.best_params_}")
print(f"Best cross-validation RMSE for Males_Migrants: {-random_search_males.best_score_:.2f}")

best_model_male_migrants = random_search_males.best_estimator_
```

```
Fitting 3 folds for each of 30 candidates, totalling 90 fits

--- RandomizedSearchCV for male migrants Complete ---

Best parameters for Males_Migrants: {'max_depth': 19, 'max_features': 'log2', 'min_samples_leaf': 2, 'min_samples_split': 6, 'n_esti
Best cross-validation RMSE for Males_Migrants: 47111.93
```

Performing hyperparameter optimization for female migration prediction using randomized search with time series cross-validation to find optimal Random Forest parameters

```
y_train_females = df_transformed[df_transformed['Year'] <= 2020]['Female_Migrants'].copy()

rf_regressor = RandomForestRegressor(random_state=42)
tscv = TimeSeriesSplit(n_splits=3)

random_search_females = RandomizedSearchCV(estimator=rf_regressor, cv=tscv, **random_search_parameters)

random_search_females.fit(X_train_encoded, y_train_females)

print("\n--- RandomizedSearchCV for female migrants Complete ---")

print(f"\nBest parameters for Females_Migrants: {random_search_females.best_params_}")
print(f"Best cross-validation RMSE for Females_Migrants: {-random_search_females.best_score_:.2f}")

best_model_female_migrants = random_search_females.best_estimator_
```

```
Fitting 3 folds for each of 30 candidates, totalling 90 fits

--- RandomizedSearchCV for female migrants Complete ---

Best parameters for Females_Migrants: {'max_depth': 19, 'max_features': 'log2', 'min_samples_leaf': 2, 'min_samples_split': 6, 'n_es
Best cross-validation RMSE for Females_Migrants: 43019.77
```

Results

Evaluating optimized Random Forest models on 2024 test data with gender-specific predictions, calculating performance metrics, and ensuring non-negative migration values

```
y_test_males = df_transformed[df_transformed['Year'] == 2024]['Male_Migrants'].copy()
y_test_females = df_transformed[df_transformed['Year'] == 2024]['Female_Migrants'].copy()

y_pred_males = best_model_male_migrants.predict(X_test_encoded)
y_pred_females = best_model_female_migrants.predict(X_test_encoded)

y_pred_males = np.maximum(0, y_pred_males) if np.any(y_pred_males < 0) else y_pred_males
y_pred_females = np.maximum(0, y_pred_females) if np.any(y_pred_females < 0) else y_pred_females

predicted_total_migrants = y_pred_males + y_pred_females

print("\nPerformance for Males_Migrants (Test Set):")
mae_males_pred = mean_absolute_error(y_test_males, y_pred_males)
```

```
rmse_males_pred = np.sqrt(mean_squared_error(y_test_males, y_pred_males))
r2_males_pred = r2_score(y_test_males, y_pred_males)
print(f"  MAE: {mae_males_pred:,.2f}")
print(f"  RMSE: {rmse_males_pred:,.2f}")
print(f"  R2: {r2_males_pred:.4f}")

print("\nPerformance for Females_Migrants (Test Set):")
mae_females_pred = mean_absolute_error(y_test_females, y_pred_females)
rmse_females_pred = np.sqrt(mean_squared_error(y_test_females, y_pred_females))
r2_females_pred = r2_score(y_test_females, y_pred_females)
print(f"  MAE: {mae_females_pred:,.2f}")
print(f"  RMSE: {rmse_females_pred:,.2f}")
print(f"  R2: {r2_females_pred:.4f}")
```

```
Performance for Males_Migrants (Test Set):
  MAE: 2,443.26
  RMSE: 21,860.38
  R2: 0.9622

Performance for Females_Migrants (Test Set):
  MAE: 2,469.36
  RMSE: 25,173.13
  R2: 0.9361
```

Comparing test and training data distributions to assess model generalization performance across gender-specific migration patterns and data variability metrics

```
print("Data Distribution Analysis:")

print("\nMale Migrants:")
print(f"Test data migration levels are {(y_test_males.mean()/y_train_males.mean()-1)*100:.1f}% higher than training")
print(f"Test data variability is {(y_test_males.std()/y_train_males.std()-1)*100:.1f}% higher than training")

print("\nFemale Migrants:")
print(f"Test data migration levels are {(y_test_females.mean()/y_train_females.mean()-1)*100:.1f}% higher than training")
print(f"Test data variability is {(y_test_females.std()/y_train_females.std()-1)*100:.1f}% higher than training")
print("\nThis indicates the model successfully generalizes to increased migration volumes.")
```

```
Data Distribution Analysis:

Male Migrants:
Test data migration levels are 53.9% higher than training
Test data variability is 25.7% higher than training

Female Migrants:
Test data migration levels are 49.9% higher than training
Test data variability is 23.5% higher than training

This indicates the model successfully generalizes to increased migration volumes.
```

Analyzing feature importance from optimized Random Forest models to identify key predictors for male and female migration patterns through visualization and ranking

```
feature_importance_males = pd.DataFrame({
    'feature': X_train_encoded.columns,
    'importance': best_model_male_migrants.feature_importances_
}).sort_values('importance', ascending=False)

feature_importance_females = pd.DataFrame({
    'feature': X_train_encoded.columns,
    'importance': best_model_female_migrants.feature_importances_
}).sort_values('importance', ascending=False)

print("Plotting Top 10 Most Important Features:")

fig, axes = plt.subplots(1, 2, figsize=(20, 8))

sns.barplot(data=feature_importance_males.head(10), x='importance', y='feature', ax=axes[0])
axes[0].set_title('Feature Importance: Male Migration Prediction', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Importance', fontsize=12)
axes[0].set_ylabel('Features', fontsize=12)

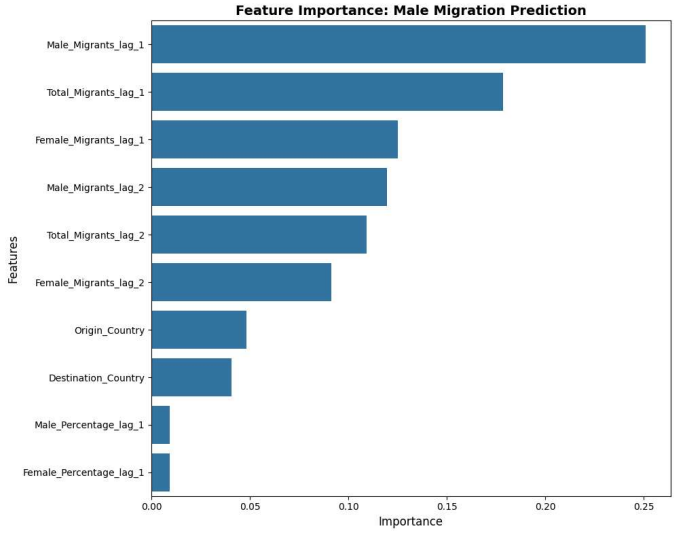
sns.barplot(data=feature_importance_females.head(10), x='importance', y='feature', ax=axes[1])
axes[1].set_title('Feature Importance: Female Migration Prediction', fontsize=14, fontweight='bold')
axes[1].set_xlabel('Importance', fontsize=12)
axes[1].set_ylabel('Features', fontsize=12)

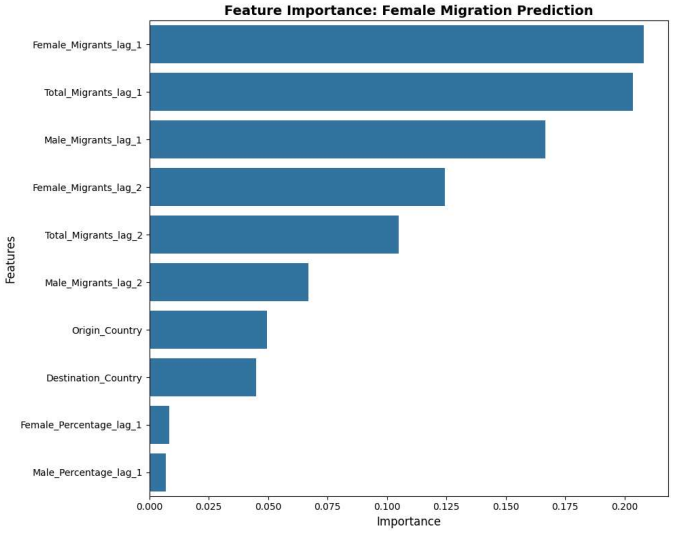
plt.tight_layout()
plt.show()
```

```
print("Top 10 Most Important Features for Male Migration:")
print(feature_importance_males.head(10).to_string(index=False))

print("\nTop 10 Most Important Features for Female Migration:")
print(feature_importance_females.head(10).to_string(index=False))
```

```
Plotting Top 10 Most Important Features:
```






```
Top 10 Most Important Features for Male Migration:
               feature  importance
    Male_Migrants_lag_1    0.250945
   Total_Migrants_lag_1    0.178562
  Female_Migrants_lag_1    0.125209
    Male_Migrants_lag_2    0.119489
   Total_Migrants_lag_2    0.109192
  Female_Migrants_lag_2    0.091304
         Origin_Country    0.048159
    Destination_Country    0.040770
  Male_Percentage_lag_1    0.009305
Female_Percentage_lag_1    0.009225

Top 10 Most Important Features for Female Migration:
               feature  importance
  Female_Migrants_lag_1    0.207914
   Total_Migrants_lag_1    0.203367
    Male_Migrants_lag_1    0.166550
  Female_Migrants_lag_2    0.124291
   Total_Migrants_lag_2    0.104898
    Male_Migrants_lag_2    0.066940
         Origin_Country    0.049472
    Destination_Country    0.044926
Female_Percentage_lag_1    0.008419
  Male_Percentage_lag_1    0.007123
```

Converting regression prediction outputs into categorical variables by applying the same ranking transformations used in feature engineering to create binary classification flags for popular destinations, migration routes, and gender dominance patterns

And Evaluating classification performance of predicted migration patterns by comparing actual vs predicted categorical flags for popular destinations, migration routes, and gender dominance using confusion matrices and classification reports

```
df_test_preds = pd.DataFrame({
    'Year': X_test['Year'],
    'Destination_Country': X_test['Destination_Country'],
    'Predicted_Males': y_pred_males,
    'Predicted_Females': y_pred_females,
    'Predicted_Total': predicted_total_migrants
}, index=X_test.index)

predicted_destination_totals = df_test_preds.groupby(['Destination_Country', 'Year'])['Predicted_Total'].sum().reset_index()
predicted_destination_totals['Is_Popular_Destination'] = predicted_destination_totals.groupby('Year')['Predicted_Total'].transform(
    lambda x: x.rank(pct=True, ascending=True) >= 0.90
).astype(int)
```

```
df_test_preds = df_test_preds.merge(
    predicted_destination_totals[['Destination_Country', 'Year', 'Is_Popular_Destination']],
    on=['Destination_Country', 'Year'],
    how='left'
)

df_test_preds['Is_Popular_Migration_Route'] = df_test_preds.groupby('Year')['Predicted_Total'].transform(
    lambda x: x.rank(pct=True, ascending=True) >= 0.90
).astype(int)

df_test_preds['Is_Male_Dominated'] = (df_test_preds['Predicted_Males'] > df_test_preds['Predicted_Females']).astype(int)

actual_flags_2024 = df_transformed[df_transformed['Year'] == 2024].copy()
actual_flags_2024 = actual_flags_2024.set_index(X_test_encoded.index)

actual_is_popular_destination = actual_flags_2024['Is_Popular_Destination']
actual_is_popular_route = actual_flags_2024['Is_Popular_Migration_Route']
actual_is_male_dominated = actual_flags_2024['Is_Male_Dominated']

print("--- Classification Report for Is_Popular_Destination (Test Set) ---")
print(classification_report(actual_is_popular_destination, df_test_preds['Is_Popular_Destination']))
print("\nConfusion Matrix for Is_Popular_Destination (Test Set):")
cm_pop_dest = confusion_matrix(actual_is_popular_destination, df_test_preds['Is_Popular_Destination'])
plt.figure(figsize=(10, 6))
sns.heatmap(
    cm_pop_dest,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Not Popular', 'Popular'],
    yticklabels=['Not Popular', 'Popular']
)
plt.title('Confusion Matrix: Is_Popular_Destination Prediction')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
plt.show()

print("\n--- Classification Report for Is_Popular_Migration_Route (Test Set) ---")
print(classification_report(actual_is_popular_route, df_test_preds['Is_Popular_Migration_Route']))
print("\nConfusion Matrix for Is_Popular_Migration_Route (Test Set):")
cm_pop_route = confusion_matrix(actual_is_popular_route, df_test_preds['Is_Popular_Migration_Route'])
plt.figure(figsize=(10, 6))
sns.heatmap(
    cm_pop_route,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Not Popular', 'Popular'],
    yticklabels=['Not Popular', 'Popular']
)
plt.title('Confusion Matrix: Is_Popular_Migration_Route Prediction')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
plt.show()

print("\n--- Classification Report for Is_Male_Dominated (Test Set) ---")
print(classification_report(actual_is_male_dominated, df_test_preds['Is_Male_Dominated']))
print("\nConfusion Matrix for Is_Male_Dominated (Test Set):")
cm_male_dom = confusion_matrix(actual_is_male_dominated, df_test_preds['Is_Male_Dominated'])
plt.figure(figsize=(10, 6))
sns.heatmap(
    cm_male_dom,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Female Dominated', 'Male Dominated'],
    yticklabels=['Female Dominated', 'Male Dominated']
)
plt.title('Confusion Matrix: Is_Male_Dominated Prediction')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
plt.show()
```